Abstract

This project focuses on the design and development of a motion simulator control system using multiple types of motors and drivers. The system integrates game telemetry data from a racing simulator into physical motion through stepper motors, AC servo motors, and DC motors. Initially, stepper motors were tested using an Arduino platform, followed by implementation of an industrial AC servo motor controlled via an ESP32 microcontroller. Later, a high-power DC motor driver with RS-485 communication was incorporated. Finally, the system was interfaced with a simulation game using motion control software to produce realistic actuator movements. A prototype motion platform using five stepper motors was also developed.

Introduction

Motion simulators are widely used in gaming, training systems, and virtual reality environments to replicate real-world physical movements. These systems rely on actuators controlled by real-time data from simulation software.

The objective of this project is to develop a scalable motion control system capable of driving multiple motor types based on telemetry data from a racing game.

Project Objective

- To develop a motor control system capable of responding to real-time simulation data
- To test different motor technologies for motion simulation
- To integrate hardware with game telemetry software
- To build a prototype motion platform

Hardware Components Used

Stepper Motor System

- NEMA 17 Stepper Motors
- TMC2208 Stepper Motor Drivers
- Arduino Boards (Uno / Mega 2560)

AC Servo System

- AC Servo Motor: 80ST-M02430
- Servo Driver: SDD01LA08F
- ESP32 Development Board

DC Motor System

- DC Motor Driver: D-AIS4815A

- RS-485 Communication Interface
- AIMotor Debug Software

Prototype Platform

- ATmega2560 Microcontroller
- Five NEMA 17 Motors
- Multiple TMC2208 Drivers

Software Used

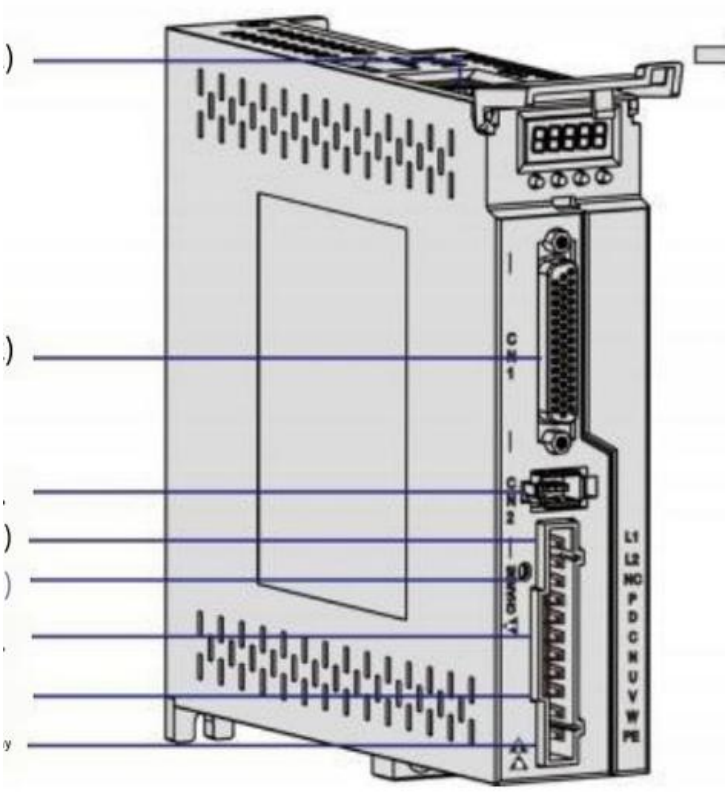
- Arduino IDE
- ESP32 Development Environment
- AIMotor Debug Software
- FlyPT Mover Motion Software
- Live for Speed (LFS) Racing Simulator

System Development Methodology

Phase 1 — Stepper Motor Testing

The project began with controlling a NEMA 17 stepper motor using an Arduino board and TMC2208 driver. This stage validated the basic motion control concept and ensured reliable step and direction signal generation.

Phase 2 — AC Servo Motor Implementation



After successful stepper control, the system was upgraded to an industrial AC servo motor (80ST-M02430) with its dedicated driver (SDD01LA08F).

Key tasks completed:

- Driver parameter configuration for external control mode
- Establishing communication between ESP32 and servo driver
- Generating control signals from ESP32
- Verifying motor response to commands

Connect CN1 To DB25 pin (which as colour pin out)

Connect CN2 to Motor Encoder(inbuilt pin to motor)

Connect power supply to motor power cable (it has L,N,U,V,W,PE etc pins)

Connect L pin to phase of power cable(Red colour) N pin to neutral (black) pin

DB44 connector pinout :

Yellow ------ Pulse+/step+

Blue -------- Direction +

Green ------ Pulse-

Black -------- Direction-

Connection to ESP32:

DB44 pin	**ESP32 pin**
Pulse+/step+	3.3v
Pulse-/step-	GPIO 2 pin
Direction+	3.3v
Direction-	GPIO 4 pin

For encoder values connect rs485 cable to CN4/CN3(ethernet type port)

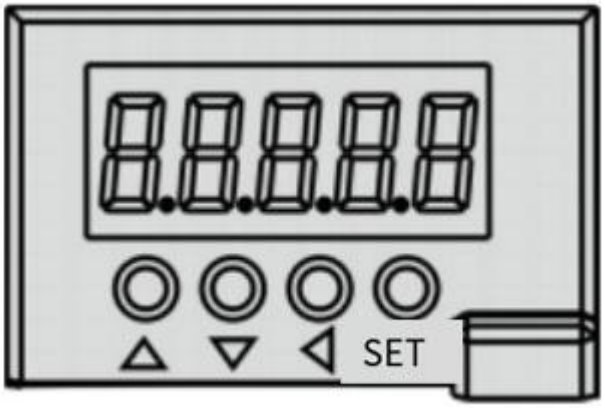
Connect to ESP32

RS485 PIN	**ESP32 PIN**
Vcc	3.3v
Gnd	Gnd
TX2	TX
RX2	RX

BASIC PARAMETERS SET TO AC MOTOR DRIVER:

Navigate to FA parameters :

When you on the motor it will display {r 0}



Display should like this

To reach the FA parameters

Press < (back) button twice then you will get dF parameter

Press ^ (up) button you will get FA parameter

When you press M(set) button you will enter into FA parameters

PARAMETER VALUES:

FA04 ------ 0

FA11 ------ 1000

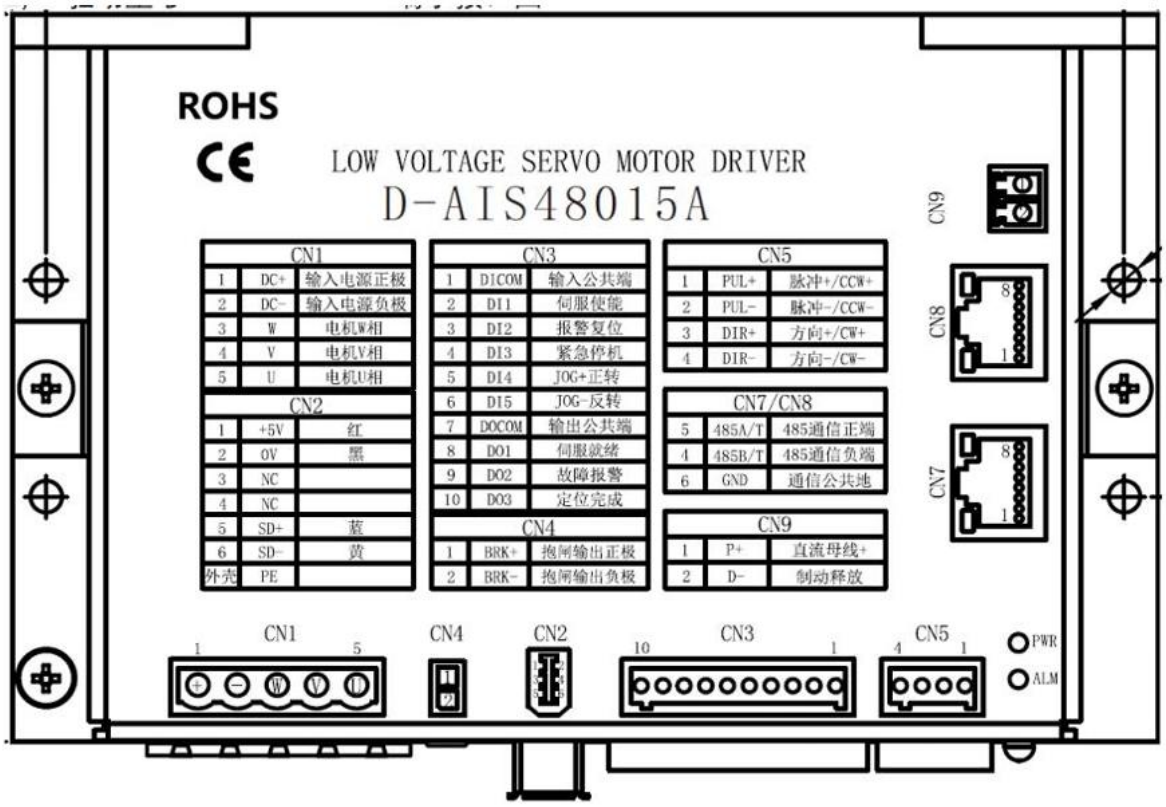
FA14 ------ 0

FA15 ------ 0

FA23 ------- 3000

The motor successfully responded to external commands issued by the ESP32 controller.

Phase 3 — DC Motor Driver Integration



A high-power DC motor driver (D-AIS4815A) was introduced.

Steps performed:

1. Communication established using RS-485 interface
2. AIMotor Debug Software used for configuration
3. Default driver parameters set for external command control
4. Successful motor operation verified

CN1 for power Supply to motor and driver

Pin 1	DC+ (24V-48v) better to provide >39v
Pin 2	DC-
Pin 3	W pin from motor
Pin 4	V pin from motor
Pin 5	U pin from motor

CN4 is connected to motor which is in black and red colored pin

CN2 is connect to motor encoder

CN5 is used to communicate between ESP32 to Driver

ESP32 CONNECTION:

CN5 PINS	ESP32 PINS
Pin 1	3.3v
Pin 2	GPIO 2
Pin 3	3.3V
Pin 4	GPIO 4

ENCODER CONNECTION is as same as AC Motor connections

PARAMETERS are default values which are given by provider if there is any zig zag in parameters then just factory set the H_parameters Section then it will works

GAME CODE :

```
/*
 Motion Simulator Controller


*/


#include <Arduino.h>


// ===== PINS =====
#define STEP_PIN 2   // Geen Colour Pin
#define DIR_PIN  4   // black colour pin // yellow and blue colour pins connect to 3.3v pin in esp32
#define EN_PIN   21  // optional pin need not required

// ===== TRAVEL SETTINGS =====
#define MAX_STROKE 40000L
#define CENTER_POS (MAX_STROKE / 2)

// ===== MOTION LIMITS =====
#define MAX_SPEED 15000.0f     // steps/sec
#define ACCEL     40000.0f     // steps/sec²
#define DEAD_BAND 5            // stop jitter

// ===== SIGNAL TIMEOUT =====
#define SIGNAL_TIMEOUT 500     // ms
```

```
// ===== TIMER =====
hw_timer_t *timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

// ===== STATE =====
volatile long currentPos = CENTER_POS;
volatile long targetPos  = CENTER_POS;
volatile float speedSPS  = 0;
volatile bool stepState  = false;

unsigned long lastDataTime = 0;

// ======================================================
// STEP GENERATION ISR (NO DELAYS)
// ======================================================
void IRAM_ATTR onTimer()
{
 portENTER_CRITICAL_ISR(&timerMux);

 long error = targetPos - currentPos;

 if (abs(error) > DEAD_BAND)
 {
  bool dir = (error > 0);
  digitalWrite(DIR_PIN, dir);

  stepState = !stepState;
  digitalWrite(STEP_PIN, stepState);

  if (stepState)  // count only on rising edge
   currentPos += dir ? 1 : -1;
```

```
  }

  portEXIT_CRITICAL_ISR(&timerMux);
}

// ======================================================
// MOTION UPDATE — ACCEL + DECEL CONTROL
// ======================================================
void updateMotion(float dt)
{
  long error = targetPos - currentPos;

  if (abs(error) <= DEAD_BAND)
  {
    speedSPS = 0;
    timerAlarm(timer, 0, true, 0);
    return;
  }

  float distance = abs(error);

  // braking distance
  float brakeDist = (speedSPS * speedSPS) / (2.0f * ACCEL);

  // DECELERATION
  if (brakeDist >= distance)
  {
    speedSPS -= ACCEL * dt;
    if (speedSPS < 0) speedSPS = 0;
  }
  else
  {
```

```
    // ACCELERATION
    speedSPS += ACCEL * dt;
    if (speedSPS > MAX_SPEED) speedSPS = MAX_SPEED;
  }

  if (speedSPS < 1) speedSPS = 1;

  uint32_t interval = (uint32_t)(1000000.0f / speedSPS);

  timerAlarm(timer, interval, true, 0);
}

// ================================================
// READ FlyPT BINARY STREAM
// ================================================
void readFlyPT()
{
  while (Serial.available() >= 2)
  {
    uint8_t low  = Serial.read();
    uint8_t high = Serial.read();

    uint16_t value = (high << 8) | low;

    long newTarget = map(value, 0, 65535, 0, MAX_STROKE);

    portENTER_CRITICAL(&timerMux);
    targetPos = constrain(newTarget, 0, MAX_STROKE);
    portEXIT_CRITICAL(&timerMux);

    lastDataTime = millis();
  }
```

```
}

// ================================================
// FAILSAFE — SIGNAL LOSS
// ================================================
void checkSignalTimeout()
{
  if (millis() - lastDataTime > SIGNAL_TIMEOUT)
  {
    targetPos = CENTER_POS;  // return to safe position
  }
}

// ================================================
// SETUP
// ================================================
void setup()
{
  pinMode(STEP_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);
  pinMode(EN_PIN, OUTPUT);

  digitalWrite(EN_PIN, LOW);

  Serial.begin(115200);

  // 1 MHz timer
  timer = timerBegin(1000000);
  timerAttachInterrupt(timer, &onTimer);
  timerAlarm(timer, 1000, true, 0);

  lastDataTime = millis();
```

```
}

// ================================================
// MAIN LOOP
// ================================================
void loop()
{
 readFlyPT();
 checkSignalTimeout();

 static uint32_t lastTime = millis();
 uint32_t now = millis();

 if (now - lastTime >= 5)   // 200 Hz control loop
 {
   float dt = (now - lastTime) / 1000.0f;
   lastTime = now;

   updateMotion(dt);
 }
}
```

AC MOTOR ENCODER CODE:

```
#include <ModbusMaster.h>

#define PUL_PIN 2
#define DIR_PIN 4
#define RX_PIN 16
#define TX_PIN 17

ModbusMaster node;

void setup() {
```

```
  Serial.begin(115200);

  // Initialize RS485 exactly as the Siheng driver demands
  Serial2.begin(57600, SERIAL_8N2, RX_PIN, TX_PIN);
  node.begin(1, Serial2);

  pinMode(PUL_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);

  Serial.println("System Ready. Telemetry Dashboard with Error Tracking Starting...");
  delay(2000);
}

void loop() {
  Serial.println("\n>>> Moving Forward... >>>");
  // Assuming Common Anode wiring based on your previous tests
  digitalWrite(DIR_PIN, LOW);

  // Move the motor
  for (int i = 0; i < 20000; i++) {
    digitalWrite(PUL_PIN, HIGH);
    delayMicroseconds(25);
    digitalWrite(PUL_PIN, LOW);
    delayMicroseconds(25);
  }

  // Stop and read the full dashboard
  readFullTelemetry();
  delay(1000);

  Serial.println("\n<<< Moving Reverse... <<<");
  digitalWrite(DIR_PIN, HIGH);
```

```
  // Move back the other way
  for (int i = 0; i < 20000; i++) {
    digitalWrite(PUL_PIN, HIGH);
    delayMicroseconds(25);
    digitalWrite(PUL_PIN, LOW);
    delayMicroseconds(25);
  }

  // Stop and read the full dashboard again
  readFullTelemetry();
  delay(1000);
}

// ========================================
//     FULL TELEMETRY DASHBOARD FUNCTION
// ========================================
// ========================================
//     FULL TELEMETRY DASHBOARD FUNCTION
// ========================================
void readFullTelemetry() {
  int16_t rpm = 0;
  float torque = 0;
  int32_t actualPos = 0;
  int32_t cmdPos = 0;
  int32_t errorPos = 0;
  float currentAmps = 0;

  Serial.println("====================================");

  // --- CHUNK 1: Read Speed and Torque ---
  uint8_t result1 = node.readHoldingRegisters(0x0B00, 3);
```

```
if (result1 == node.ku8MBSuccess) {
  rpm = (int16_t)node.getResponseBuffer(0);
  torque = (int16_t)node.getResponseBuffer(2)  / 10.0;
} else {
  Serial.print("Chunk 1 Error: 0x"); Serial.println(result1,  HEX);
}

// --- CHUNK 2: Read Positions (Shortened to 8 registers for speed) ---
uint8_t result2 = node.readHoldingRegisters(0x0B07,  8);
if (result2 == node.ku8MBSuccess) {
  uint16_t actLow  = node.getResponseBuffer(0);
  uint16_t actHigh = node.getResponseBuffer(1);
  actualPos = (int32_t)((actHigh << 16) | actLow);

  uint16_t cmdLow  = node.getResponseBuffer(6);
  uint16_t cmdHigh = node.getResponseBuffer(7);
  cmdPos = (int32_t)((cmdHigh << 16) | cmdLow);

  // FIX: Let the ESP32 calculate the exact error perfectly!
  errorPos = cmdPos - actualPos;
} else {
  Serial.print("Chunk 2 Error: 0x"); Serial.println(result2,  HEX);
}

// --- CHUNK 3: Read Phase Current ---
uint8_t result3 = node.readHoldingRegisters(0x0B18,  1);
if (result3 == node.ku8MBSuccess) {
  // FIX: Added (int16_t) so negative current reads correctly
  currentAmps = (int16_t)node.getResponseBuffer(0)  / 100.0;
} else {
  Serial.print("Chunk 3 Error: 0x"); Serial.println(result3,  HEX);
}
```

```
  // --- PRINT THE DASHBOARD ---
  Serial.print("1.  Speed (RPM):          "); Serial.println(rpm);
  Serial.print("2.  Actual Position:       "); Serial.println(actualPos);
  Serial.print("3.  Commanded Position:    "); Serial.println(cmdPos);
  Serial.print("4.  Following Error:       "); Serial.println(errorPos);
  Serial.print("5.  Motor Load/Torque (%): "); Serial.println(torque);
  Serial.print("6.  Motor Current (Amps):  "); Serial.println(currentAmps);
  Serial.println("=====================================\n");
}
```

DC MOTOR ENCODER CODE:

```
#include <ModbusMaster.h>

#define PUL_PIN 2
#define DIR_PIN 4
#define RX_PIN 16
#define TX_PIN 17

ModbusMaster node;

void setup() {
  Serial.begin(115200);

  // Initialize RS485 for the Siheng driver
  Serial2.begin(57600, SERIAL_8N2, RX_PIN, TX_PIN);
  node.begin(1, Serial2);

  pinMode(PUL_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);

  Serial.println("System  Ready. Telemetry Dashboard Starting...");
  delay(2000);
```

```
}

void loop() {
 // ========================================
 // 1. ANNOUNCE DIRECTION & MOVE FORWARD
 // ========================================
 Serial.println("\n>>> Moving Forward at 1500 RPM... >>>");
 digitalWrite(DIR_PIN, LOW);

 // 40,000 pulses at 19us delay = ~1.5 seconds of movement at 1500 RPM
 for (long i = 0; i < 40000; i++) {
  digitalWrite(PUL_PIN, HIGH);
  delayMicroseconds(9);
  digitalWrite(PUL_PIN, LOW);
  delayMicroseconds(9);
 }

 // ========================================
 // 2. STOP & PRINT DASHBOARD
 // ========================================
 Serial.println("||| STOPPED |||");
 readFullTelemetry();
 delay(3000); // Wait 2 seconds so you can read it

 // ========================================
 // 3. ANNOUNCE DIRECTION & MOVE REVERSE
 // ========================================
 Serial.println("\n<<< Moving Reverse at 1500 RPM... <<<");
 digitalWrite(DIR_PIN, HIGH);

 for (long i = 0; i < 40000; i++) {
  digitalWrite(PUL_PIN, HIGH);
```

```
    delayMicroseconds(9);
    digitalWrite(PUL_PIN, LOW);
    delayMicroseconds(9);
  }

  // ==========================================
  // 4. STOP & PRINT DASHBOARD
  // ==========================================
  Serial.println("|||  STOPPED  |||");
  readFullTelemetry();
  delay(3000); // Wait 2 seconds so you can read it
}

// ==========================================
//      FULL TELEMETRY DASHBOARD FUNCTION
// ==========================================
void readFullTelemetry() {
  int16_t rpm = 0;
  float torque = 0;
  int32_t actualPos = 0;
  int32_t cmdPos = 0;
  int32_t errorPos = 0;
  float currentAmps = 0;

  Serial.println("====================================");

  // --- CHUNK 1: Read Speed and Torque ---
  uint8_t result1 = node.readHoldingRegisters(0x0B00,  3);
  if (result1 == node.ku8MBSuccess) {
    rpm = (int16_t)node.getResponseBuffer(0);
    torque = (int16_t)node.getResponseBuffer(2)  / 10.0;
  } else {
```

```
  Serial.print("Chunk 1 Error: 0x"); Serial.println(result1, HEX);
}

// --- CHUNK 2: Read Positions ---
uint8_t result2 = node.readHoldingRegisters(0x0B07, 8);
if (result2 == node.ku8MBSuccess) {
  uint16_t actLow  = node.getResponseBuffer(0);
  uint16_t actHigh = node.getResponseBuffer(1);
  actualPos = (int32_t)((actHigh << 16) | actLow);

  uint16_t cmdLow  = node.getResponseBuffer(6);
  uint16_t cmdHigh = node.getResponseBuffer(7);
  cmdPos = (int32_t)((cmdHigh << 16) | cmdLow);

  // Calculate the exact error perfectly
  errorPos = cmdPos - actualPos;
} else {
  Serial.print("Chunk 2 Error: 0x"); Serial.println(result2, HEX);
}

// --- CHUNK 3: Read Phase Current ---
uint8_t result3 = node.readHoldingRegisters(0x0B18, 1);
if (result3 == node.ku8MBSuccess) {
  // Keep the (int16_t) fix so reverse current shows a correct negative number!
  currentAmps = (int16_t)node.getResponseBuffer(0) / 100.0;
} else {
  Serial.print("Chunk 3 Error: 0x"); Serial.println(result3, HEX);
}

// --- PRINT THE DASHBOARD ---
Serial.print("1. Speed (RPM):        "); Serial.println(rpm); // Will be 0 since motor is stopped
Serial.print("2. Actual Position:    "); Serial.println(actualPos);
```

```
  Serial.print("3. Commanded Position:    "); Serial.println(cmdPos);
  Serial.print("4. Following Error:       "); Serial.println(errorPos);
  Serial.print("5. Motor Load/Torque (%): "); Serial.println(torque);

  Serial.print("6. Motor Current (Amps):  "); Serial.println(currentAmps);
  Serial.println("====================================\n");
}
```

Phase 4 — Game Integration

The system was connected to a racing simulation environment:

Game: Live for Speed (LFS)
Motion Software: FlyPT Mover

Process:

1. Game telemetry data transmitted to FlyPT Mover
2. Motion cues calculated by software
3. Commands sent to motor controllers
4. Motors executed movements based on game dynamics

Both AC and DC motors successfully operated according to simulation data.

Phase 5 — Motion Platform Prototype

A physical prototype simulator platform was developed using:

- ATmega2560 microcontroller
- Five NEMA 17 stepper motors
- TMC2208 drivers

This prototype demonstrates multi-actuator motion control capability.

Working Principle

1. Simulation game generates real-time vehicle motion data
2. FlyPT Mover processes telemetry into actuator commands
3. Commands are transmitted to motor controllers
4. Microcontrollers generate control signals

5. Motors produce physical movement corresponding to game events

Results

- Successful control of stepper, AC servo, and DC motors
- Reliable communication between software and hardware
- Real-time response to game telemetry
- Functional multi-motor prototype platform

Challenges Faced

- Driver parameter configuration for external control
- Communication setup for RS-485 devices
- Integration between different hardware platforms
- Synchronizing motor response with real-time game data

Conclusion

The project successfully demonstrates the development of a versatile motion simulator control system using multiple motor technologies. Through progressive implementation and testing, the system evolved from basic stepper motor control to industrial-grade AC and DC motor integration with real-time game telemetry. The prototype confirms the feasibility of building a full motion simulator platform using these techniques.